Reductions and Extension-Based Proofs

Kayman Brusse
kayman@cs.toronto.edu
Department of Computer Science
University of Toronto
Canada

Faith Ellen
faith@cs.toronto.edu
Department of Computer Science
University of Toronto
Canada

ABSTRACT

In the theory of distributed computing, the notion of a *reduction* is a common tool for proving impossibility results. If task $\mathcal{T}$ reduces to task $\mathcal{S}$, and $\mathcal{T}$ is impossible to solve, then so is $\mathcal{S}$. Extension-based proofs demonstrate the impossibility of solving a task in a wait-free manner by constructing an infinite execution. It is known that extension-based proofs are limited in power: there is no extension-based proof of the impossibility of a wait-free protocol in the NIS model for k-set agreement among $n > k \geq 2$ processes.

We introduce augmented extension-based proofs, which generalize extension-based proofs by providing a new type of query. For a general class of reductions, we prove that, if $\mathcal{T}$ reduces to $\mathcal{S}$, and $\mathcal{T}$ has an augmented extension-based proof that it is impossible to solve in the NIS model, then so does $\mathcal{S}$. Since there is also no augmented extension-based proof of the impossibility of a wait-free protocol in the NIS model for k-set agreement among $n > k \geq 2$ processes, it follows that there are no (augmented) extension-based proofs of the impossibility of wait-free protocols in the NIS model for a number of other distributed computing problems.

CCS CONCEPTS

• **Theory of computation → Problems, reductions and completeness**; **Distributed algorithms**; **Distributed computing models**; **Interactive proof systems**.

KEYWORDS

reductions; impossibility proofs; valency arguments; renaming

ACM Reference Format:
Kayman Brusse and Faith Ellen. 2021. Reductions and Extension-Based Proofs. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC '21), July 26–30, 2021, Virtual Event, Italy*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3465084.3467906

1 INTRODUCTION

Throughout the field of theoretical computer science, the notion of a reduction from one problem to another is used extensively. One of the most common applications of a reduction is to transform an existing impossiblity result from one task to another. If a task $\mathcal{T}$ reduces to task $\mathcal{S}$, and $\mathcal{T}$ is impossible to solve, then so is $\mathcal{S}$.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '21, July 26–30, 2021, Virtual Event, Italy
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8548-0/21/07…$15.00
https://doi.org/10.1145/3465084.3467906

In particular, reductions have been used to show that various distributed computing tasks are impossible to solve. Most commonly, reductions are from consensus, where processes must agree on one value to output from among their input values. The impossibility of a wait-free protocol for consensus in an asynchronous system where processes may crash was first proved by Fischer, Lynch, and Paterson [11]. Their proof introduced the *valency argument*, a technique that constructs an infinite execution by repeatedly extending a finite execution. The k-set agreement task is a generalization of consensus where processes can output at most k different values. The impossibility of a wait-free protocol for k-set agreement was first proven independently by Borowsky and Gafni [6], Herlihy and Shavit [16], and Saks and Zaharoglou [18]. These proofs use employ sophisticated tools from combinatorial topology.

Alistarh, Aspnes, Ellen, Gelashvili and Zhu [2, 3] introduced the class of *extension-based proofs*, a generalization of valency arguments, which also show the impossibility of a wait-free protocol for a task by repeatedly extending a finite execution to construct an infinite execution. They proved that there is no extension-based proof of the impossibility of a wait-free protocol for k-set agreement among $n > k \geq 2$ processes in both the *non-uniform iterated immediate snapshot* (NIIS) model and the *non-uniform iterated snapshot* (NIS) model, where processes communicate through an infinite sequence of snapshot objects. Since there are wait-free simulations of the NIS and NIIS models in the standard asynchronous shared memory model where processes communicate through read/write registers, this implies that there is also no extension-based proof of the impossibility of a wait-free protocol for k-set agreement among $n > k \geq 2$ processes in the standard asynchronous shared memory model. They conjectured there are other tasks, such as $(2n-2)$-renaming [9], which are known to be impossible to solve by wait-free anonymous protocols, but have no extension-based proofs that they are impossible to solve by wait-free anonymous protocols in the NIS or NIIS models.

Just as reductions can be used to transform impossibility *results* from one task to another, it is natural to ask whether reductions can be used to transform extension-based impossibility *proofs* from one task to another. Answering this question would further our understanding of the extension-based proofs and their limitations.

Our goal was to show that, if a task $\mathcal{T}$ reduces to task $\mathcal{S}$, and $\mathcal{T}$ has an extension-based proof of its impossibility in the NIS model, then so does $\mathcal{S}$. Although we were unable to do this, we were able to prove that this claim holds for a slightly larger class of proofs and a general class of reductions. Our class of *augmented extension-based proofs* gives the prover access to more information about a protocol. We also show that there is no augmented extension-based proof of the impossibility of solving k-set agreement among $n > k \geq 2$ processes in a wait-free manner in the NIS model. It follows that,

if task $\mathcal{T}$ reduces to k-set agreement, then there is no augmented extension-based proof of the impossibility of solving $\mathcal{T}$ in a wait-free manner in the NIS model. There are several known reductions involving k-set agreement [6, 13, 14].

An *extension-based proof* is an interaction between a prover and a protocol that claims to solve a task in a wait-free manner. Initially, the prover only knows the initial configurations of the protocol. To obtain more information, the prover can submit *queries* that ask about the behaviour of the protocol when certain processes take steps from configurations known to the prover. The prover uses this information with the aim of either constructing an infinite execution, or discovering a configuration where the protocol fails to satisfy the specifications of the task. If the prover succeeds, then the protocol does not solve the task in a wait-free manner, and the prover is said to *win* the interaction. A task has an extension-based proof if there is a prover that wins against *any* protocol.

In an *augmented* extension-based proof, the prover is given additional power in the form of a new type of query. This query allows the prover to obtain more information about the protocol. We present extension-based proofs and augmented extension-based proofs in more detail in Section 4. In the full version of the paper, we prove that there is no augmented extension-based proof of the impossibility of solving k-set agreement among $n > k \geq 2$ processes in a wait-free manner in the NIS model, following the technique of Alistarh, Aspnes, Ellen, Gelashvili and Zhu [3]. We highlight the main differences between these two proofs in Section 5.

If $\mathcal{T}$ and $\mathcal{T}'$ are tasks, their *composition* $\mathcal{T}' \circ \mathcal{T}$ is a task that requires a protocol to first solve $\mathcal{T}$ and then solve $\mathcal{T}'$, where outputs from the solution of the first task serve as inputs to the second task. Intuitively, task $\mathcal{T}$ *reduces* to task $\mathcal{S}$ if is possible to construct a protocol that solves $\mathcal{T}$ by using a protocol that solves $\mathcal{S}$. In the simplest kind of reduction, every input to $\mathcal{T}$ can be locally converted to an input to $\mathcal{S}$ and, after solving $\mathcal{S}$, each process can perform local computation to determine an output that satisfies the specifications of $\mathcal{T}$. But reductions can be more complicated: processes might communicate with one another to determine input values for $\mathcal{S}$, they might solve many instances of $\mathcal{S}$, and they might communicate with one another to determine output values that satisfy the specifications of $\mathcal{T}$. In this paper, we consider reductions which involve solving only one instance of $\mathcal{S}$, but allow processes to communicate with one another beforehand and afterwards. Such a reduction can be represented as $\mathcal{R}_2 \circ \mathcal{S} \circ \mathcal{R}_1$, where $\mathcal{R}_1$ and $\mathcal{R}_2$ are tasks that are solvable in the NIS model. Given any input vector x, the processes first solve the task $\mathcal{R}_1$, then solve the task $\mathcal{S}$, and finally solve the task $\mathcal{R}_2$. The resulting output vector y must satisfy the specifications of $\mathcal{T}$. This is illustrated in Figure 1. These reductions are more general than many-one reductions, but less general than Turing reductions.

To make use of a reduction in the NIS model, we need to *compose* the protocols for solving these tasks. Composing protocols is straightforward in most other models, such as the *iterated snapshot* (IS) model. However, this is not the case in the NIS model. In Section 6, we explain where the difficulty arises and describe an algorithm for composing two NIS protocols.

Finally, we prove that if task $\mathcal{T}$ reduces to task $\mathcal{S}$, and $\mathcal{T}$ has an augmented extension-based impossibility proof in the NIS model, then $\mathcal{S}$ also has an augmented extension-based impossibility proof in the NIS model. To do this, we describe how an augmented extension-based prover can win in its interaction against any NIS protocol A that claims to solve $\mathcal{S}$. Suppose $\mathcal{R}_2 \circ \mathcal{S} \circ \mathcal{R}_2$ is a reduction from $\mathcal{T}$ to $\mathcal{S}$. If A_1 and A_2 are known NIS protocols that solve $\mathcal{R}_1$ and $\mathcal{R}_2$, respectively, and A solves S, then $A_2 \circ A \circ A_1$ solves $\mathcal{T}$. Our prover *simulates* an interaction between this composed protocol and an augmented extension-based prover that wins against any protocol that claims to solve $\mathcal{T}$. The only way for the prover to win in this simulated interaction is to prove that A does not solve $\mathcal{S}$. Our prover decides which queries it will submit in its interaction with A based on the queries that it sees in this simulation. We present a high-level description of our prover's strategy in Section 7. The complete strategy appears in the full version of the paper.

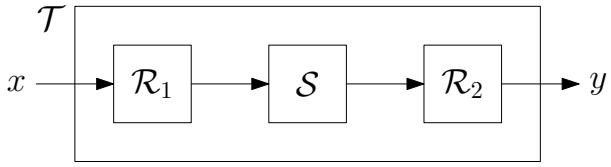


Figure 1: An example of a reduction $(\mathcal{R}_1, \mathcal{R}_2)$ from the task $\mathcal{T}$ to the task $\mathcal{S}$

2 RELATED WORK

The *iterated snapshot* (IS) model and the *non-uniform iterated snapshot* (NIS) model are closely related to the *iterated immediate snapshot* (IIS) model, introduced by Borowsky and Gafni [8], and the *non-uniform iterated immediate snapshot* (NIIS) model, introduced by Hoest and Shavit [17]. In the IIS and NIIS models, the reachable configurations of a protocol have a natural representation using combinatorial topology. In the IS and NIS models, the reachable configurations of a protocol can be represented using graphs. In terms of computability, a task is solvable in any of these models if and only if it is solvable in the standard asynchronous shared memory model where processes communicate by reading from and writing to shared registers: Any protocol in the standard asynchronous shared memory model can be modified to use a snapshot object by having processes perform a `scan` instead of a `read` and ignore any extra information. There is a wait-free implementation of a snapshot object from registers, due to Afek, Attiya, Dolev, Gafni, Merritt, and Shavit [1]. The IS and NIS models can be simulated in the snapshot model by having processes append their `update` operations and ignore any extra information they see when performing a `scan`. Similarly, the IIS and NIIS models can be simulated in the *immediate* snapshot model. Borowsky and Gafni [7] gave a wait-free implementation of an immediate snapshot object from snapshot objects. In addition, Borowsky and Gafni [8] gave a non-blocking simulation of a snapshot object in the NIIS model. This simulation also works in the NIS model. In addition, using Köonig's lemma, they proved that if a task is solvable, then it is solvable by a bounded wait-free protocol. It follows that, if a task is solvable in the NIIS (or NIS) model, then it is solvable in the IIS (or IS) model.

Herlihy, Kozlov, and Rajsbaum [15, Chapter 9.4.1] gave a definition of a reduction for the IIS model that is similar to our definition

for the NIS model. Their definition is more general, as it allows several instances of a protocol for $\mathcal{S}$ to be used when constructing a protocol for $\mathcal{T}$. In practice, reductions, such as those surveyed by Gafni [12], are described in the standard asynchronous shared memory model. There are reductions from 2-set agreement to approximate agreement on a number of graphs, including cycles of length at least four [4, 10]. Borowsky and Gafni [6] presented reductions from k-set agreement to the k-leader election task, from k-leader election to the k-test-and-set task, and vice versa. Likewise, Gafni, Rajsbaum, and Herlihy [14] presented reductions between $(2n-2)$-renaming and the weak symmetry breaking task. All of these reductions satisfy our definition. They also gave a reduction from weak symmetry breaking to $(n-1)$-set agreement, but it uses two instance of $(n-1)$-set agreement, so it does not satisfy our definition. However, Herlihy, Kozlov, and Rajsbaum [15, Chapter 9.4.2] later gave a simpler version that does satisfy our definition. Interestingly, Gafni, Rasjbaum, and Herlihy [14] proved that there is no reduction from $(n-1)$-set agreement to weak symmetry breaking and, hence, to $(2n-2)$-renaming. The impossibility of a wait-free protocol for $(2n-2)$-renaming when n is a prime power is due to Castañeda and Rajsbaum [9].

Alistarh, Ellen, and Rybicki [4] proved that there is no extension-based proof of the impossibility of solving approximate agreement on the 4-cycle in the NIS model. Note that this does not follow from our results, since there is no known reduction from 4-cycle agreement to k-set agreement.

Attiya, Castañeda, and Rajsbaum [5] studied the limitations of valency arguments in the IIS model. In their proofs, the prover learns the set of values that are output by processes in final configurations reachable from a given configuration, but it does not learn which value is output by any specific process. They prove their class does not contain proofs that $(n-1)$-set agreement and weak symmetry breaking are impossible to solve in the IIS model. They state that the same is true for $(2n-2)$-renaming, by reduction. Note that, in the IS and IIS models, extension-based proofs are too powerful, since a prover can perform exhaustive search to discover every reachable configuration of a protocol.

3 MODEL

The *non-uniform iterated snapshot* (NIS) model consists of $n \geq 2$ processes $p_1, \ldots, p_n$ and an infinite sequence $S_1, S_2, \ldots$ of shared single-writer snapshot objects, each with n components. The initial value of each component is blank. The *initial state* of process p_i is (i, x_i), where i is its identifier and x_i is its input. Each process accesses each snapshot object at most twice, in order, starting with S_1. After a process outputs a value, it is *terminated* and accesses no further snapshot objects. In the NIS model, we assume that a process is only terminated after performing a `scan`. This is without loss of generality, because a `scan` does not change the contents of shared memory and, so, does not affect any other process.

A *protocol* in the NIS model specifies what values a process uses to `update` the snapshot objects it accesses, how `update` and `scan` operations affect its local state, when it outputs, and what value it outputs. A *step* of a protocol consists of an access of a snapshot object by a process followed the local computation by that process necessary for it to obtain its new state and decide whether to terminate. When proving impossibility results, it is convenient to assume that each process remembers its entire history. Such a protocol is called *full-information*. A full-information protocol in the NIS model can be described in a standard form: The first time p_i accesses S_h, it performs `update`(s_i) to set $S_h[i]$ equal to its current state s_i. Its new state is $s_i 1$, where the appended bit indicates that p_i last performed an `update`. The second time p_i accesses S_h, it performs a `scan`. Its new state, s_i', is (i, v), where v is the response from the `scan`. Since each process `updates` its component of a snapshot object with its current state the step before it `scans`, its new state contains all of its previous states. Next, p_i consults a *decision function*, δ, from the set of possible states to the set of possible output values and the special symbol $\perp$. If $\delta(s_i') \neq \perp$ then p_i outputs $\delta(s_i)$ and is *terminated*. If $\delta(s_i) = \perp$ then p_i becomes poised to access the next snapshot object. Thus, a full-information protocol in the NIS model is completely specified by its decision function δ.

A *configuration* of a protocol consists of the state of each process, which processes are terminated, the outputs of all terminated processes, and the contents of each snapshot object. Since at most one `update` is performed on each component of each snapshot object, a configuration of a full-information NIS protocol is completely specified by the states of all the processes in the configuration and the value of the decision function on these states. A process is *active* in a configuration if it is not terminated. A configuration is *final* if it has no active processes. If C is a configuration and p_i is a process that is active in C, then Cp_i denotes the configuration that results when p_i takes a step of the protocol from configuration C. A *schedule* of a protocol from C is a finite or infinite sequence of processes $\alpha = \alpha_1, \alpha_2, \ldots$ such that there is a sequence of configurations $C = C_0, C_1, \ldots$ where process α_i is active in C_{i-1} and $C_i = C_{i-1}\alpha_i$ for all α_i in α. If α is a schedule from C of length ℓ, then $C\alpha$ denotes the configuration C_ℓ. A P-only schedule from C is a schedule in which only processes in P appear. Each finite schedule from an initial configuration results in a *reachable* configuration. A protocol is *wait-free* if each process is terminated within a finite number of steps. Thus, a protocol is wait-free if and only if it has no infinite schedule. If α is a schedule from an initial configuration, the *h'th round of α* consists of all the `update` and `scan` operations performed on the snapshot object S_h during the execution of α. A process *terminates in round h* if it is terminated after performing its `scan` of S_h. In the NIS model, processes can terminate in different rounds of a schedule. In contrast, for every protocol in the IS model, there exists a positive integer t such that, in every schedule of the protocol from an initial configuration, every process terminates in round t.

A *task* $\mathcal{T}$ consists of a finite set of input vectors, I, a finite set of output vectors, O, and a map $T : I \to 2^O$ that specifies, for each input vector $x \in I$, a non-empty set $T(x) \subseteq O$ of valid output vectors. An initial configuration of a protocol *has input vector x* if each process p_i has input x_i in that configuration. Similarly, a final configuration of a protocol *has output vector y* if each process p_i has output y_i in that configuration. Suppose that A is a wait-free protocol that has an initial configuration with input vector x for each $x \in I$. Furthermore, suppose that $y \in T(x)$ for each final configuration of A that has output vector y and is reachable from

an an initial configuration of A that has input vector x. Then we say that the protocol *solves* task $\mathcal{T}$.

If $\mathcal{T}$ and $\mathcal{T}'$ are tasks with maps T and T', respectively, their *composition* $\mathcal{T}' \circ \mathcal{T}$ is a task that has the same set of input vectors as $\mathcal{T}$ and the same set of output vectors as $\mathcal{T}'$. Moreover, for each input vector x, the set of valid output vectors is $T' \circ T(x) = \cup\{T'(y) \mid y \in T(x)\}$. Formally, a pair of tasks, $(\mathcal{R}_1, \mathcal{R}_2)$, is a *reduction* from $\mathcal{T}$ to $\mathcal{S}$ if there are NIS protocols that solve $\mathcal{R}_1$ and $\mathcal{R}_2$ and, for each input vector x of $\mathcal{T}$, each output vector for x that satisfies the specifications of the task $\mathcal{R}_2 \circ \mathcal{S} \circ \mathcal{R}_1$ also satisfies the specifications of $\mathcal{T}$, i.e., $R_2 \circ S \circ R_1(x) \subseteq T(x)$, where R_1, R_2, S, and T are the maps for $\mathcal{R}_1$, $\mathcal{R}_2$, $\mathcal{S}$, and $\mathcal{T}$, respectively.

4 AUGMENTED EXTENSION-BASED PROOFS

An augmented extension-based proof is a generalization of an extension-based proof in which the prover is allowed to ask an additional type of query. We begin by presenting a brief description of extension-based proofs. More detailed explanations can be found in the original papers in which they were introduced [2, 3].

Extension-Based Proofs

An *extension-based proof* is an interaction between a prover and a (supposedly) wait-free protocol that claims to solve a task. The prover attempts to refute this claim by constructing an infinite schedule, or by reaching a configuration where the protocol violates the task specification. The interaction proceeds in *phases*, beginning with phase 1. In each phase φ, the prover maintains a finite schedule $\alpha(\varphi)$ that it has constructed so far and a finite set of configurations it has reached during the phase. Each of the configurations it reaches during phase φ must be of the form $C_0\alpha(\varphi)\beta$ for some initial configuration C_0 and some schedule β from $C_0\alpha(\varphi)$. Initially, the schedule $\alpha(1)$ is empty and the prover has reached the initial configurations of the protocol.

During each phase φ, the prover can submit *queries* to obtain more information about the protocol. A *query* (C, q) made by the prover specifies a configuration C it has reached during the phase and a process q that is active in C. The protocol responds to the query with the configuration Cq that results from process q taking one step in the protocol from C. As a result of this query, the prover reaches configuration Cq. Note that, since $C = C_0\alpha(\varphi)\beta$ for some initial configuration C_0 and some schedule β from $C_0\alpha(\varphi)$, it follows that $Cq = C_0\alpha(\varphi)\beta'$, where $\beta' = \beta q$ is also a schedule from $C_0\alpha(\varphi)$. If the prover reaches a configuration where the protocol violates the specifications of the task, then the protocol is incorrect and the prover *wins*.

A *chain of queries* is a sequence of queries $(C_1, q_1), (C_2, q_2), \ldots$, where, for each $i \geq 1$, the configuration C_{i+1} is the response $C_i q_i$ of the previous query. If the prover is able to submit an *infinite* chain of queries, then it has constructed an infinite schedule of the protocol from an initial configuration (consisting of the schedule from an initial configuration to C_1, followed by the sequence $q_1, q_2, \ldots$). This demonstrates that the protocol is not wait-free, and the prover wins.

An *output query* (C, P, y) made by the prover specifies a configuration C it has reached, a set of processes P, and an output value y. If there is a P-only schedule of the protocol from C in which some process in P outputs the value y, then the protocol responds with such a schedule. Otherwise, the protocol responds None. If the protocol responds with a schedule, the prover can use this schedule to decide which queries it will submit in the future. By submitting an output query for each possible output value y, the prover can determine which values are output in schedules of the protocol from configuration C. Notice that the prover does not reach any new configurations as the result of an output query, but it can always submit a chain of queries corresponding to the schedule returned by the protocol.

After submitting finitely many chains of queries and output queries in phase φ, the prover must end the phase. The prover chooses a configuration C that it reached during phase φ, where $C = C_0\alpha(\varphi)\beta$ for some initial configuration C_0 and some non-empty schedule β, and sets $\alpha(\varphi + 1) = \alpha(\varphi)\beta$. At the beginning of phase $\varphi + 1$, the prover has reached all configurations $C_0'\alpha(\varphi + 1)$, where C_0' is an initial configuration in which every process that appears in $\alpha(\varphi+1)$ has the same input as it does in C_0. Note that configurations $C = C_0\alpha(\varphi+1)$ and $C_0'\alpha(\varphi+1)$ are indistinguishable to every process that appears in the schedule $\alpha(\varphi + 1)$.

If the interaction lasts for an infinite number of phases, then protocol is not wait-free and the prover wins. If, at the end of some phase, the prover chooses a configuration C in which every process is terminated and the values output by processes satisfy the specifications of the task, then the prover *loses*. A task has an extension-based impossibility proof if there is a prover that wins in its interaction against *any* protocol that claims to solve the task.

Augmented Extension-Based Proofs

In an *augmented extension-based proof*, output queries are replaced with *assigment queries*. An assignment query (C, P, f) made by the prover specifies a configuration C it has reached, a set of processes P, and an assignment f from a set of processes $Q \subseteq P$ to possible output values. A configuration C' *satisfies* f if, for each process $q \in Q$, the process q is terminated in C' with output $f(q)$. If there is a P-only schedule from C that leads to a configuration C' satisfying f, then the protocol responds with such a schedule. Otherwise the protocol responds None. An augmented extension-based prover is allowed to perform a finite number of assignment queries in each phase.

By replacing output queries with assignment queries, we allow the prover to obtain more information about the protocol. Using assignment queries, the prover can ask if there is a schedule in which multiple values are output and it can specify which processes output which values. Any task that has an extension-based impossibility proof also has an augmented extension-based impossibility proof. To see why, observe that an augmented extension-based prover can use assignment queries to simulate any output query (C, P, y). Specifically, the prover submits the assignment queries (C, P, g_p), for all $p \in P$, where $g_p : \{p\} \to \{y\}$ maps process p to the output value y. If there is a P-only schedule from C in which some process in $q \in P$ outputs y, then the protocol will respond with such a schedule to the assignment query (C, P, g_q). On the other hand, if there is no such schedule, then the protocol will respond None to all of these assignment queries.

5 THE NON-EXISTENCE OF AUGMENTED EXTENSION-BASED PROOFS OF THE IMPOSSIBILITY OF k-SET AGREEMENT

There is no extension-based proof of the impossibility of solving k-set agreement among $n > k \geq 2$ processes in the NIS model [3]. Likewise, there is no augmented extension-based proof of the impossibility of solving k-set agreement among $n > k \geq 2$ processes in the NIS model. In this section, we describe the main similarities and differences between the proofs of these two results. We refer to proof of the first result as the *original* proof.

In both proofs, an NIS protocol specified by the decision function δ is represented by a sequence of graphs $\mathbb{G}_t$, for $t \geq 0$. The graph $\mathbb{G}_t$ represents all configurations reachable from initial configurations of the protocol by t-round schedules. Each n-vertex clique $\sigma \subseteq \mathbb{G}_t$ represents one such configuration C. The *subdivision* $\chi(\sigma, \delta)$ of σ is a graph that represents all the configurations of the protocol reachable by 1-round schedules from C. It is shown in the original proof that $\mathbb{G}_{t+1}$ is equal to the *subdivision* $\chi(\mathbb{G}_t, \delta)$ of $\mathbb{G}_t, \delta)$, which is the union of $\chi(\sigma, \delta)$ for all n-vertex cliques σ in $\mathbb{G}_t$.

The original proof studies the how distance between two subgraphs $\mathbb{A}$ and $\mathbb{B}$ in $\mathbb{G}_t$ compares to the distance between the subdivisions of those two subgraphs, $\chi(\mathbb{A}, \delta)$ and $\chi(\mathbb{B}, \delta)$, in $\mathbb{G}_{t+1}$. In our proof, we define the *active distance* between subgraphs $\mathbb{A}$ and $\mathbb{B}$ in $\mathbb{G}_t$ to be the minimal number of edges between *non-terminated* vertices in any path from $\mathbb{A}$ to $\mathbb{B}$ in $\mathbb{G}_t$. We show that the active distance between $\chi(\mathbb{A}, \delta)$ and $\chi(\mathbb{B}, \delta)$ in $\mathbb{G}_{t+1}$ is exactly double the active distance between $\mathbb{A}$ and $\mathbb{B}$ in $\mathbb{G}_t$. Intuitively, this is because each edge between non-terminated vertices in a path between $\mathbb{A}$ and $\mathbb{B}$ in $\mathbb{G}_t$ is replaced by two edges between non-terminated vertices in $\mathbb{G}_{t+1}$.

Both proofs define an adversarial protocol that is able to win in the interaction against any prover. The adversary maintains a *partial* specification of a protocol, δ, an integer, $t \geq 0$, and some invariants. The specification of δ is defined using the graph $\mathbb{G}_t$.

When the prover submits a query in the first phase, the adversary can increase t and specify more of δ so that it can respond to the prover while maintaining the invariants. In the original proof, one of the invariants guarantees that any two vertices in $\mathbb{G}_t$ that are terminated with different output values are sufficiently far apart in $\mathbb{G}_t$. This implies that, at the end of phase 1, no clique in $\mathbb{G}_t$ contains vertices that output different values.

It is impossible to maintain this invariant when the prover is allowed to submit assignment queries. To see why, notice that the adversary would have to respond `None` to any assignment query that asks if there is a schedule in which two or more distinct values are output. By doing so, the adversary implicitly tells the prover that the protocol δ solves *consensus*. Since consensus has an extension-based impossibility proof, there is a strategy the prover can use to win against such an adversary.

In our proof, the adversary maintains a different set of invariants, most of which rely on the notion of active distance. Essentially, we allow the adversary to terminate vertices that belong to the same clique σ in $\mathbb{G}_t$ with two different output values, but we require that the active distance between σ and any terminated vertex w that outputs a value different from either of those two values is at least 1. This means that when the adversary subdivides $\mathbb{G}_t$ to create $\mathbb{G}_{t+1}$, the active distance between σ and w will increase by a factor of two. At the end of phase 1, the adversary subdivides several times, after which it is possible to partition the terminated vertices in $\mathbb{G}_t$ into sets such that (1) the vertices in each set output at most two different values, and (2) the active distance between two different sets of terminated vertices in $\mathbb{G}_t$ is at least 4. For each set V of terminated vertices in this partition, the adversary chooses a value b that is output by some vertex in V. Then the adversary terminates additional vertices so that the vertices in V are surrounded by vertices that all output value b.

After this has been done, the vertices in each clique of $\mathbb{G}_t$ still output at most two different values. Moreover, if a clique contains at least one non-terminated vertex, then all terminated vertices in that clique output the same value. Effectively, this is the same situation as the end of phase 1 in the original proof. The rest of the argument is the same.

In an *anonymous NIS protocol*, the steps taken by a process do not depend on its identifier. More formally, the initial state of a process is its input value and the result of a `scan` of a snapshot object is the multi-set of the nonblank values stored in its components. Moreover, the value a process uses to `update` a snapshot object, when a process is terminated, and what it outputs when it is terminated are only functions of its state. Note that, in full-information protocols, these three properties are ensured by definition. Since anonymous NIS protocols are a subset of all NIS protocols, it might be easier to prove that there is no anonymous NIS protocol that solves some task $\mathcal{T}$.

The prover in an (augmented) extension-based proof can refute the claim that a full-information NIS protocol is anonymous by reaching two configurations, C and C', such that some process q has terminated in C and some process q' has terminated with a different output value in C' or has not terminated in C' , but the state of q in configuration C is the same as the state of q' in configuration C'. By maintaining an additional invariant, namely that the partially specified protocol has the same decision value for all processes having the same state, we are able to show that there is also no augmented extension-based proof of the impossibility of an anonymous protocol that solves k-set agreement among $n > k \geq 2$ processes in the NIS model.

6 COMPOSING NIS PROTOCOLS

Let A_1 and A_2 be NIS protocols such that each output from A_1 is a valid input for A_2. We describe how to construct the *composition* $A_2 \circ A_1$ of these protocols, which is an NIS protocol that simulates a schedule of A_1 from the same initial configuration, followed by a schedule of A_2, where the outputs from the first schedule serve as the inputs to the second schedule. To avoid confusion, we let S'_h, S''_h, and S_h denote the h'th snapshot object accessed by processes in schedules of A_1, A_2, and $A_2 \circ A_1$, respectively.

In the special case when A_1 is an IS protocol, there is some positive integer t such that each process is terminated when it has taken exactly $2t$ steps. This property makes it straightforward to compose A_1 and A_2: First, processes simulate a schedule of A_1, using snapshot object S_h in place of S'_h, for $h = 1, \ldots, t$. After taking exactly $2t$ steps, instead of terminating with output y_i, each

process p_i sets its local state to (i, y_i), which is its state in an initial configuration of A_2 when it receives y_i as input. Next, the processes simulate a schedule of A_2, using snapshot object $S_{t+\ell}$ in place of S''_ℓ to simulate round ℓ.

If A_1 is an NIS protocol, this straightforward approach for composing A_1 and A_2 does not work: When processes simulate a schedule of A_1, they might finish in different rounds. If each process simply starts to simulate a schedule of A_2 immediately afterwards, different processes could access different snapshot objects when simulating the first round of A_2. Instead, processes can use estimate vectors to simulate the rounds of A_2. Estimate vectors were introduced by Borowsky and Gafni [8] in their non-blocking simulation of a snapshot object in the NIIS model. In their protocol, an *estimate vector* is a potential output from a scan of the snapshot object. Each process repeatedly modifies its estimate vector based on the values it sees in the estimate vectors of other processes. When a process *finalizes* its estimate vector, it uses this estimate vector as a valid response from a scan operation.

In our protocol, each process p_i simulates steps of process p_i in protocol A_1, followed by steps of process p_i in protocol A_2. It also maintains a sequence of estimate vectors, where its ℓ'th estimate vector is an estimate for the response of its scan of S''_ℓ in the simulated schedule of A_2. After each scan it performs in the composed protocol, it can modify any estimate vectors that it has not yet finalized. Even if a process has not started simulating its scan of S''_ℓ (or is still simulating A_1), it may have to modify its ℓ'th estimate vector because of information it receives from other processes that have already started simulating their accesses to S''_ℓ.

We begin with a high level description of $A_2 \circ A_1$, followed by a detailed description. A process begins its simulation of the composed protocol by directly simulating its update and scan of each snapshot object S'_h, in order, using S_h. It obtains the simulated response to its scan of S'_h from the response of its scan of S_h by considering each component j of the response to be blank unless it sees that the corresponding process p_j was simulating A_1 when it performed its update to S_h. Then it uses this simulated response to update its simulated state. After a process has finished simulating A_1, it modifies its first estimate vector to include its initial state for the simulation of A_2 and then tries to finalize it. To finalize its ℓ'th estimate vector, for any $\ell \geq 1$, the process repeatedly performs an update to the next snapshot object in the composed protocol (to contain ℓ and its first ℓ estimate vectors) followed by a scan, until the response from the scan satisfies certain conditions. When it has finalized its ℓ'th estimate vector, the process uses this estimate vector as its response from its scan of S''_ℓ in its simulation of A_2. Based on this response, the process may decide to terminate its simulation of A_2. If not, it modifies its $(\ell+1)$'st estimate vector to include its new state in the simulation of A_2 and tries to finalize its $(\ell+1)$'st estimate vector. Note that a process may perform many updates and scans in the composed protocol to simulate its scan in each round of A_2.

We assume that A_1 and A_2 are full information protocols. However, our description of the composed protocol is not expressed as a full information protocol, to make it easier to understand and facilitate the proof of correctness. Since any protocol can be transformed into a full information protocol without changing the number of steps it performs (but possibly making it harder to understand), it is still possible to compose the resulting protocol with another protocol using the same approach.

Formal Description of $A_2 \circ A_1$

For any two n-component vectors u and u', we say that $u \preceq u'$ if and only if for all $1 \leq j \leq n$, either $u[j]$ is blank or $u[j] = u'[j]$. Consider any schedule from a configuration in which all components of a single-writer snapshot object S are blank, each process updates S at most once, and $F_1, \ldots, F_t$ are the responses to the scans of S, in the order they were performed. Then $F_1 \preceq \cdots \preceq F_t$. More generally, $\preceq$ is a partial order on n-component vectors.

Throughout the protocol $A_2 \circ A_1$, each process p_i locally maintains an infinite sequence E_i of estimate vectors, each with n components. Initially, every component of every estimate vector is blank. At any point during the protocol, only components of the estimate vectors in a finite prefix of this sequence contain nonblank values. We denote the ℓ'th estimate vector in this sequence by $E_{i,\ell}$ and component $1 \leq j \leq n$ of this estimate vector by $E_{i,\ell}[j]$. Process p_i also locally maintains a number r_i, which is 0 while it is simulating A_1 and is the round in A_2 it is simulating, otherwise. We attach a superscript h to a local variable of process p_i to denote its value immediately before p_i performs its update to S_h. Initially, E_i^1 is the empty sequence and $r_i^1 = 0$.

The protocol will ensure that, in any schedule from an initial configuration, if the j'th components of the ℓ'th estimate vectors of two processes are both not blank, then they contain the same value.

When process p_i is simulating round $r_i^h = \ell$ in A_2, it updates each snapshot object S_h it accesses with ℓ followed by its first ℓ estimate vectors, $E_{i,1}^h, \ldots, E_{i,\ell}^h$.

While simulating steps of A_1, process p_i uses s_i to locally record its simulated state. If x_i is the input to p_i in $A_2 \circ A_1$, then $s_i^1 = (i, x_i)$. To simulate its update to S'_h, it performs update(s_i^h) to the snapshot object S_h and appends 1 to s_i (to indicate that it has last performed an update). To simulate its scan of S'_h, process p_i performs a scan of S_h and receives an n component vector $v_{i,h}$ in response. When computing the response $v'_{i,h}$ to the simulated scan, it ignores the contents of $v_{i,h}[j]$, replacing it with blank if it sees that process p_j was simulating A_2 when it performed its update to S_h. However, it uses this information to update its estimate vectors. Specifically, if $v_{i,h}[j]$ contains $r_j^h = \ell > 0$ followed by the first ℓ estimate vectors of E_j^h, then process p_i updates its first ℓ estimate vectors so that they contain all the nonblank values in those estimate vectors, i.e., for all $1 \leq \ell' \leq \ell$ and all components $1 \leq j' \leq n$, if $E_{i,\ell'}[j']$ is blank and $E_{j,\ell'}^h[j']$ is not blank, then this value is copied to $E_{i,\ell'}[j']$. Next, process p_i updates its simulated state in A_1 to be $s_i^{h+1} = (i, v'_{i,h})$. Finally, p_i consults the decision function δ_1 for the protocol A_1. If $\delta_1(s_i^{h+1}) = \perp$, then p_i simply continues to round $h+1$ of its simulation of A_1. If $\delta_1(s_i^{h+1}) = y_i \neq \perp$, then y_i is its output in the simulated execution of A_1. In this case, p_i increments r_i, so $r_i^{h+1} = 1$, and sets $E_{i,1}[i]$ to be $F_{i,0} = (i, y_i)$.

When p_i performs a scan of S_h while it is simulating round $\ell = r_i^h$ in A_2, it uses the response $v_{i,h}$ to determine how to update the estimate vectors it has not yet finalized and to determine

whether it can finalize its ℓ'th estimate vector. Specifically, for each component $1 \le j \le n$, if $v_{i,h}[j]$ contains $r_j^h \ge \ell$ followed by the first r_j^h estimate vectors of E_j^h, then p_i updates its ℓ'th through r_j^h'th estimate vectors so that they contain all the nonblank values in those estimate vectors, i.e., for all $\ell \le \ell' \le r_j^h$ and all components $1 \le j' \le n$, if $E_{j,\ell'}^h[j']$ is not blank, then $E_{i,\ell'}[j']$ is assigned this value. Process p_i does not finalize its ℓ'th estimate vector if,

- p_i changes its ℓ'th estimate vector using information from $v_{i,h}$, i.e., for some component j, $v_{i,h}[j]$ contains $r_j^h \ge \ell$ followed by the first r_j^h estimate vectors of E_j^h, where $E_{j,\ell}^h \not\preceq E_{i,\ell}^h$,
- p_i sees that some other process updated S_h when that process was simulating A_1 or an earlier round of A_2, i.e., for some component j, $v_{i,h}[j]$ contains s_j^h or contains $r_j^h < \ell$ followed by the first r_j^h estimate vectors of E_j^h, or
- p_i sees that some other process that was also simulating round ℓ of A_2, but its ℓ'th estimate vector was different than p_i's, i.e. for some component j, $v_{i,h}[j]$ contains ℓ followed by the first ℓ estimate vectors of E_j^h, where $E_{j,\ell}^h \ne E_{i,\ell}^h$.

In all these cases, p_i simply `updates` S_{h+1} in its next step of the composed protocol. Otherwise, for each component $1 \le j \le n$,

- $v_{i,h}[j]$ is blank or
- $v_{i,h}[j]$ contains $r_j^h \ge \ell$ followed by the first r_j^h estimate vectors of E_j^h and
  - either $r_j^h = \ell$ and $E_{j,\ell}^h = E_{i,\ell}^h$
  - or $r_j^h > \ell$ and $E_{j,\ell}^h \preceq E_{i,\ell}^h$.

In this case, p_i finalizes its ℓ'th estimate vector to be $F_{i,\ell} = E_{i,\ell}^h$ and consults the decision function δ_2 for the protocol A_2. If $\delta_2(i, F_{i,\ell}) = z_i \ne \perp$, then p_i outputs z_i and is terminated. If $\delta_2(i, F_{i,\ell}) = \perp$, then p_i needs to simulate another round of A_2. It begins simulating this round by incrementing r_i, so $r_i^{h+1} = \ell + 1$, and setting $E_{i,\ell+1}[i] = (i, F_{i,\ell})$. Note that r_i is now larger than ℓ, so process p_i does not change its ℓ'th estimate vector after it has been finalized.

If A_1 and A_2 are anonymous protocols, then an anonymous version of $A_2 \circ A_1$ can be obtained by removing process identifiers from stored values and replacing estimate vectors by multi-sets. It is also necessary to define $M \preceq M'$ to mean every element of multi-set M occurs at least as many times in multi-set M'.

Overview of Correctness

By construction, no process changes any component of any of its estimate vectors to blank. There are only two ways a process changes the i'th component of its ℓ'th estimate vector from blank: it copies the value from the i'th component of the ℓ'th estimate vector of some other process or process p_i sets it to be $(i, F_{i,\ell-1})$ when it increments r_i to ℓ. Thus, if the i'th component of an ℓ'th estimate vector is not blank, it is equal to $(i, F_{i,\ell-1})$. Moreover, if $r_i^h < \ell$, then process p_i does not include its ℓ'th estimate vector when it `updates` S_h. Thus, no other process sees $(i, F_{i,\ell-1})$ during its `scan` of S_h. It follows that $E_{j,\ell}^{h+1}[i]$ is blank for all $j \ne i$.

Suppose that A_1 is a protocol that solves task $\mathcal{T}_1$ and A_2 is a protocol that solves task $\mathcal{T}_2$. We explain why the composed protocol $A_2 \circ A_1$ solves the task $\mathcal{T}_2 \circ \mathcal{T}_1$. A complete proof appears in the full version of the paper.

Let C_1 be an initial configuration of the composed protocol $A_2 \circ A_1$ and let α be a schedule of $A_2 \circ A_1$ from C_1. Since $\mathcal{T}_2 \circ \mathcal{T}_1$ and $\mathcal{T}_1$ have the same set of input vectors, C_1 is also an initial configuration of A_1. The *restriction of α to A_1* is the schedule α_1 obtained from α by removing all occurrences of each process after it has finished simulating A_1. While a process is simulating A_1, it does not use its estimate vectors to update its simulated state in A_1 and it ignores information about estimate vectors of processes simulating A_2, except to update its own estimate vectors. Thus, α_1 is a schedule of $A_2 \circ A_1$ from C_1. Moreover, since each `update` (and `scan`) of S_h that occurs in α_1 is simulating a `update` (and `scan`) of S'_h, it follows that α_1 is a schedule of A_1 from C_1 and each process p_i finishes its simulation of A_1 with output y_i during schedule α of $A_2 \circ A_1$ from C_1 if and only if p_i terminates at the end of the same round with output y_i during schedule α_1 of A_1 from C_1.

Let P be the set of processes that finish their simulation of A_1 during α from C_1 and, for each $p_i \in P$, let y_i be its simulated output from A_1. Let C_2 be any initial configuration of A_2 in which each process $p_i \in P$ has input y_i. We show that, during α, the processes in P simulate a P-only schedule of protocol A_2 from C_2. To do so, we first prove that, if process p_j finalizes its ℓ'th estimate vector after process p_i during schedule α from C_1, then $F_{i,\ell} \preceq F_{j,\ell}$. This property is used to show that the estimate vectors finalized by processes correspond to the responses of `scans` in some P-only schedule β of A_2 from C_2. Specifically, the order in which processes finalize their ℓ'th estimate vectors during schedule α from C_1 is the order in which they perform their `scans` of S''_ℓ in the schedule β of A_2 they are simulating. Borowsky and Gafni [8] proved a similar property about their non-blocking simulation of a snapshot object in the NIIS model. It follows that process p_i is terminated with output z_i during schedule α of $A_2 \circ A_1$ from C_1 if and only if p_i is terminated with output z_i during schedule β of A_2 from C_2. This implies that if z is an output vector from a schedule of $A_2 \circ A_1$ from an initial configuration with input vector x, then there is a vector y such that $y \in T_1(x)$ and $z \in T_2(y)$, where T_1 and T_2 are the maps for $\mathcal{T}_1$ and $\mathcal{T}_2$, respectively.

Finally, we prove that the schedule α from C_1 is finite. Suppose not. Let Q be the processes that take an infinite number of steps during α. Since A_1 is wait-free, the restriction α_1 of α to A_1 is finite. Hence, every process in Q is simulating A_2 for an infinite number of steps.

Suppose that some process simulates the same round of A_2 for an infinite number of steps. Let ℓ be the smallest such round. Let $Q' \subseteq Q$ be those processes that take an infinite number of steps during α, but do not finalize their ℓ'th estimate vectors. Let t be large enough such that, during schedule α from C_1, only processes in Q access S_t, all processes in Q' are simulating round ℓ of A_2 when they access S_t, and all processes in $Q - Q'$ have finalized their ℓ'th estimate vectors prior to accessing S_t. For all $h \ge t$, no process in Q' finalizes its ℓ'th estimate vector after its `scan` of S_h. It follows from the definition of $A_2 \circ A_1$ that at least one process $p_i \in Q'$ changes a blank component of its ℓ'th estimate vector using information

from $v_{i,h}$. Since components of estimate vectors are never changed back to blank, eventually the ℓ'th estimate vectors of all processes in Q' have no blank components. This is a contradiction.

Therefore, all processes in Q eventually finalize their ℓ'th estimate vectors, for all $\ell \geq 1$. This implies that, during α, the processes are simulating an infinite schedule of A_2. But this is impossible, since A_2 is wait-free.

7 REDUCTIONS AND EXTENSION-BASED PROOFS

In this section, we prove the main result of the paper:

Theorem 1. *If task $\mathcal{T}$ reduces to task $\mathcal{S}$, and there is an augmented extension-based proof of the impossibility of solving $\mathcal{T}$ in the NIS model, then there is an augmented extension-based proof of the impossibility of solving $\mathcal{S}$ in the NIS model.*

Let $(\mathcal{R}_1, \mathcal{R}_2)$ be a reduction from $\mathcal{T}$ to $\mathcal{S}$. Let $\mathscr{P}_{\mathcal{T}}$ be an augmented extension-based prover that wins against any protocol claiming to solve $\mathcal{T}$. We construct an augmented extension-based prover $\mathscr{P}$ that wins against any NIS protocol claiming to solve $\mathcal{S}$.

Fix NIS protocols A_1 and A_2 that solve $\mathcal{R}_1$ and $\mathcal{R}_2$. If a task is solvable in the NIS model, it is solvable in the IS model. Thus, we may assume, without loss of generality, that A_1 is an IS protocol, which simplifies our construction. Suppose every process terminates in round t of A_1. The prover $\mathscr{P}$ knows everything about the protocols A_1 and A_2. For example, $\mathscr{P}$ can determine the state of every process in every configuration that is reachable from an initial configuration of either A_1 or A_2. Since both protocols are wait-free, there are only a finite number of such configurations. In addition, $\mathscr{P}$ knows t.

Let A be an arbitrary NIS protocol that claims to solve task $\mathcal{S}$. Notice that, if A *did* solve $\mathcal{S}$, then $A_2 \circ A \circ A_1$ would solve $\mathcal{T}$. To decide which queries to make in its interaction with A and how to interpret the responses to its queries, the prover $\mathscr{P}$ *simulates* an interaction between $\mathscr{P}_{\mathcal{T}}$ and the composed protocol $A_2 \circ A \circ A_1$. In this simulation, $\mathscr{P}$ will respond to the queries submitted by $\mathscr{P}_{\mathcal{T}}$ to $A_2 \circ A \circ A_1$, using its *partial* knowledge of A and its complete knowledge of A_1 and A_2. Initially, $\mathscr{P}$ has only reached the initial configurations of A. Some of the queries that $\mathscr{P}_{\mathcal{T}}$ submits will only concern processes that are simulating A_1. Since $\mathscr{P}$ knows everything about A_1, it can respond to these queries immediately, without obtaining any new information about A. When $\mathscr{P}_{\mathcal{T}}$ submits queries that involve processes taking steps in the simulation of A, $\mathscr{P}$ may not have enough information about A to determine how to respond. In this case, $\mathscr{P}$ submits its own queries to A to obtain the information it needs.

Let γ be a schedule of the composed protocol $A_2 \circ A \circ A_1$ from some initial configuration C_0. Recall that the restriction of γ to A_1 is the schedule γ_1 obtained from γ by removing all occurrences of each process after it has finished simulating A_1. It is a schedule of A_1 from C_0, so it contains at most $2t$ occurrences of each process. We say that an initial configuration D_0 of A is *compatible* with configuration $C_0\gamma_1$ if the output value of every process that is terminated in $C_0\gamma_1$ is the input value of that process in D_0. The *restriction of γ to A* is the schedule γ' obtained from the restriction of γ to $A \circ A_1$ by removing the first $2t$ occurrences of each process (or all occurrences, if the process occurs less than $2t$ times). Note that γ' is a schedule of A from any initial configuration of A compatible with configuration $C_0\gamma_1$.

Throughout its simulation of $\mathscr{P}_{\mathcal{T}}$, the prover $\mathscr{P}$ will maintain the following invariant: If $\mathscr{P}_{\mathcal{T}}$ has reached configuration $C = C_0\gamma$ of $A_2 \circ A \circ A_1$, then $\mathscr{P}$ has reached configuration $D_0\gamma'$ in its interaction with A, for some initial configuration of D_0 of A compatible with $C_0\gamma_1$, where γ_1 is the restriction of γ to A_1 and γ' is the restriction of γ to A.

In each phase φ of its interaction with A, the prover $\mathscr{P}$ maintains a finite schedule $\alpha'(\varphi)$ of A from an initial configuration. The simulated prover $\mathscr{P}_{\mathcal{T}}$ also maintains a schedule $\alpha(\psi)$ of the composed protocol from an initial configuration in each phase ψ of of its simulated interaction with $A_2 \circ A \circ A_1$ such that $\alpha'(\varphi)$ is the restriction of $\alpha(\psi)$ to A.

Notice that that the current phase ψ of the simulated interaction is not necessarily the same as the current phase φ in the interaction between $\mathscr{P}$ and A. However, it will always be the case that $\varphi \leq \psi$.

At the beginning of phase 1, $\mathscr{P}$ starts with the empty schedule and has reached the initial configurations of A. In the simulated interaction, $\mathscr{P}_{\mathcal{T}}$ also begins with the empty schedule and has only reached the initial configurations of $A_2 \circ A \circ A_1$. Since every initial configuration of A is compatible with every initial configuration of A_1, the invariant holds.

Let $\varphi \geq 1$ be the current phase of the interaction between $\mathscr{P}$ and A and let $\psi \geq 1$ be the current phase in the simulated interaction. We describe the strategy $\mathscr{P}$ uses in its interaction with A by describing what $\mathscr{P}$ does when it simulates $\mathscr{P}$ submitting a query, submitting an assignment query, and ending a phase.

Responding to queries

Suppose that the prover $\mathscr{P}_{\mathcal{T}}$ submits a query (C, q) in phase ψ of the simulated interaction. The prover $\mathscr{P}$ must respond with the configuration Cq that results from q taking one step in the composed protocol from C. Since C is a configuration that $\mathscr{P}_{\mathcal{T}}$ has reached during phase ψ, there is an initial configuration C_0 of the composed protocol and a finite schedule β such that $C = C_0\alpha(\psi)\beta$. Let $\gamma = \alpha(\psi)\beta$, let γ_1 be the restriction of γ to A_1, and let γ' be the restriction of γ to A. By the invariant, $\mathscr{P}$ has reached some configuration $D_0\gamma'$ of A, where D_0 is an initial configuration of A compatible with $C_0\gamma_1$. Since $\alpha'(\varphi)$ is the restriction of $\alpha(\psi)$ to A, $\alpha'(\varphi)$ is a prefix of γ'. We consider a number of cases, depending on what step q takes in configuration C.

First suppose that q appears less than $2t - 1$ times in γ, so q is simulating a step of A_1, but does not complete its simulation of A_1. Then the prover $\mathscr{P}$ can determine the state of q in configuration Cq from its knowledge of A_1. The states of the other processes are the same in configurations C and Cq. Thus $\mathscr{P}$ can return configuration Cq to $\mathscr{P}_{\mathcal{T}}$. Because the simulation of A is not affected and the same processes have terminated in the restrictions of γ and γq to A_1, the invariant continues to hold.

Now suppose that q appears exactly $2t - 1$ times in γ, so q is simulating its final step in the simulation of A_1. As in the previous case, $\mathscr{P}$ can return configuration Cq to $\mathscr{P}_{\mathcal{T}}$. The simulation of A is not affected, so γ' is also the restriction of γq to A. However, q has terminated with some output y in the restriction of γq to A_1. Let D'_0

be the initial configuration of A in which q has input y and every other process has the same input as in configuration D_0. Then D_0' is compatible with the configuration resulting from the restriction $\gamma_1 q$ of γq to A_1. Note that q does not appear in γ', so $\mathscr{P}$ can submit a chain of queries from $D_0'\alpha'(\varphi)$ following the rest of γ' until it reaches $D_0'\gamma'$, ensuring the invariant continues to hold.

Next, suppose that q appears at least $2t$ times in γ and q is active in configuration $D_1\gamma'$. This means that when q takes a step from C in the composed protocol, it takes a step in the simulation of A. The prover $\mathscr{P}$ submits the query $(D_1\gamma', q)$ to A and receives the configuration $D_1\gamma' q$, which describes what happens when q takes this step in A. Based on this information, $\mathscr{P}$ returns the configuration $C q$ to $\mathscr{P}_\mathcal{T}$. Since $\gamma' q$ is the restriction of γq to A, the invariant continues to hold.

Finally, suppose that q is terminated in configuration $D_1\gamma'$. This means that when q takes a step in the composed protocol from C, it is simulating A_2. The input values for the simulation of A_2 are precisely the values output by processes in $D_1\gamma'$. The prover $\mathscr{P}$ uses its knowledge of A_2 and how NIS protocols are composed to determine the state of process q in configuration Cq. The prover $\mathscr{P}$ then returns the configuration Cq to $\mathscr{P}_\mathcal{T}$. The simulation of A is not affected and the invariant continues to hold.

Responding to assignment queries

Suppose that $\mathscr{P}_\mathcal{T}$ submits an assignment query (C, P, f) in phase ψ of the simulated interaction. Let $Q \subseteq P$ be the set of processes to which f assigns output values. If there is a P-only schedule from C that leads to a configuration of $A_2 \circ A \circ A_1$ which satisfies f, then $\mathscr{P}$ must respond with such a schedule. Otherwise, $\mathscr{P}$ must respond None. The prover $\mathscr{P}_\mathcal{T}$ does not reach any new configurations as a result of an assignment query, so the invariant continues to hold regardless of what $\mathscr{P}$ does.

Since C is a configuration that $\mathscr{P}_\mathcal{T}$ has reached during phase ψ, there is an initial configuration C_0 and a finite schedule β such that $C = C_0\alpha(\psi)\beta$. Let $\gamma = \alpha(\psi)\beta$, let γ_1 be the restriction of γ to A_1, and let γ' be the restriction of γ to A. By the invariant, $\mathscr{P}$ has reached some configuration $D_0\gamma'$ of A, where D_0 is an initial configuration of A compatible with $C_0\gamma_1$. Each process that appears in γ' already finished its simulation of A_1 during γ from C_0 and, hence, is terminated in $C_0\gamma_1$. Therefore, its output in $C_0\gamma_1$ is its input in D_0.

First, the prover $\mathscr{P}$ considers all P-only schedules μ_1 of A_1 from $C_0\gamma_1$ such that every process in P is terminated in $C_0\gamma_1\mu_1$. Since A_1 is wait-free, there are only finitely many such schedules. For each such schedule μ_1, consider an initial configuration D of A compatible with configuration $C_0\gamma_1\mu_1$. Every process that appears in γ' has the same input in D and D_0. Thus $\mathscr{P}$ has reached configuration $D\alpha'(\varphi)$. Since $\alpha'(\varphi)$ is the restriction of $\alpha(\psi)$ to A, and $\alpha(\psi)$ is a prefix of γ, it follows that $\alpha'(\varphi)$ is a prefix of γ'. Thus $\mathscr{P}$ can submit a chain of queries to A from $D\alpha'(\varphi)$ to reach $D\gamma'$.

Next, $\mathscr{P}$ computes all the initial configurations C_2 of A_2 such that there is a P'-only schedule from C_2 that results in configuration satisfying f, where $Q \subseteq P' \subseteq P$. Since A_2 is wait-free, there are only a finite number of such schedules. For each of these configurations C_2, consider the assignment g that maps each process in P' to its input value in C_2. The prover $\mathscr{P}$ submits the assignment query $(D\gamma', P, g)$ to A, for all choices of D and g. If A responds to one of these assignment queries with a schedule μ', then $D\gamma'\mu'$ is a configuration of A in which every process in P' is terminated with the output value specified by g. Since μ_1 is also a schedule of the composed protocol from configuration $C = C_0\gamma$, it follows that μ' is also a schedule of the composed protocol from $C\mu_1$. In configuration $C\mu_1\mu'$, every process in P' is finished simulating A and its output from this simulation is the same as its input in C_2.

Although there is a P'-only schedule from C_2 that leads to a configuration of A_2 that satisfies f, it might not be possible for processes in P' to simulate this schedule from configuration $C\mu_1\mu'$. For example, certain processes in P' may have taken steps in the simulation of A_2 during schedule γ from C_0, which restricts the schedules of A_2 that can be simulated from $C\mu_1\mu' = C_0\gamma\mu_1\mu'$. Although the schedules of A_2 that can be simulated from $C\mu_1\mu'$ do depend on the values output by processes in P' in the simulation of A, they do not depend on the particular schedule μ' that results in these values being output. Using its knowledge of A_2 and how NIS protocols are composed, $\mathscr{P}$ examines all P'-only schedules from $C\mu_1\mu'$ to see if any lead to a configuration satisfying f. Because each process in P' is simulating A_2, and A_2 is wait-free, there are only finitely many such schedules. If it finds a schedule μ_2 that leads to a configuration satisfying f, then $\mathscr{P}$ responds to $\mathscr{P}_\mathcal{T}$ with the schedule $\mu_1\mu'\mu_2$. If, for every choice of D and g, there is no such schedule then $\mathscr{P}$ responds None.

Ending phases

Suppose that the prover $\mathscr{P}_\mathcal{T}$ decides to end its current phase ψ in the simulated interaction by choosing a configuration $C = C_0\alpha(\psi)\beta$ of $A_2 \circ A \circ A_1$ that it reached during phase ψ, where β is a non-empty schedule. Then $\alpha(\psi+1) = \alpha(\psi)\beta$ and, at the beginning of phase $\psi+1$, it has reached all configurations $C_0'\alpha(\psi+1)$, where C_0' is an initial configuration in which every process that appears in $\alpha(\psi+1)$ has the same input as it does in C_0. Let $\gamma = \alpha(\psi)\beta$. By the invariant, $\mathscr{P}$ has reached configuration $D_0\gamma'$, for some initial configuration D_0 of A compatible with $C_0\gamma_1$, where γ_1 is the restriction of γ to A_1 and γ' is the restriction of γ to A. Note that configuration D_0 of A is also compatible with $C_0'\gamma_1$ for every initial configuration C_0' in which every process that appears in $\alpha(\psi+1)$ has the same input as it does in C_0.

It is possible that processes only simulate A_1 and A_2 during schedule β from $C_0\alpha(\psi)$. In this case, $\mathscr{P}$ does nothing. Because $\mathscr{P}$ has reached $D_0\gamma'$, the invariant continues to hold. Moreover, since the restrictions of $\alpha(\psi)$ and $\alpha(\psi+1)$ to A are the same, $\alpha'(\varphi)$ is the restriction of $\alpha(\psi+1)$ to A.

Otherwise, at least one process takes a step in the simulation of A during schedule β from $C_0\alpha(\psi)$. Then $\gamma' = \alpha'(\phi)\beta'$ for some non-empty schedule β'. In this case, the prover $\mathscr{P}$ ends phase φ by choosing configuration $D_0\gamma'$ and setting $\alpha'(\varphi+1) = \gamma'$, so it is the restriction of $\alpha(\psi+1)$ to A. At the beginning of phase $\varphi+1$, $\mathscr{P}$ has reached all configurations $D_0'\alpha'(\varphi+1)$, where D_0' is an initial configuration in which every process that appears in $\alpha'(\varphi+1)$ has the same input as it does in D_0. In particular, it has reached $D_0\gamma'$, so the invariant continues to hold.

Why the prover wins

By assumption, $\mathscr{P}_{\mathcal{T}}$ wins in the simulated interaction with $A_2 \circ A \circ A_1$. One way this can happen is that the prover $\mathscr{P}_{\mathcal{T}}$ reaches a configuration $C = C_0\gamma$ in its last phase ψ of the simulated interaction and the values output by processes in C violate the specifications of $\mathcal{T}$. By the invariant, during its last phase φ, the prover $\mathscr{P}$ has reached configuration $D_0\gamma'$ in its interaction with A, for some initial configuration of D_0 of A compatible with $C_0\gamma_1$, where γ_1 is the restriction of γ to A_1 and γ' is the restriction of γ to A. If the values output by processes in $D_0\gamma'$ *did* satisfy the specifications of $\mathcal{S}$, then the values output in C would satisfy the specifications of the task $\mathcal{R}_1 \circ \mathcal{S} \circ \mathcal{R}_2$. This is a contradiction, as any outputs that are valid for the composed task $\mathcal{R}_2 \circ \mathcal{S} \circ \mathcal{R}_2$ are also valid for $\mathcal{T}$. Hence, $\mathscr{P}$ reaches a configuration in its interaction with A that violates the specifications of $\mathcal{S}$.

Another way that $\mathscr{P}_{\mathcal{T}}$ can win in the simulated interaction is by submitting an infinite chain of queries. Because A_1 and A_2 are wait-free, neither of these protocols have an infinite schedule. If $\mathscr{P}_{\mathcal{T}}$ submits an infinite chain of queries, it must be that an infinite number of these queries ask about a process taking a step in its simulation of A. According to its strategy, this implies that $\mathscr{P}$ submits an infinite chain of queries in its interaction with A.

Finally, $\mathscr{P}_{\mathcal{T}}$ can win because the number of phases are infinite. When $\mathscr{P}$ responds to a query or assignment query from $\mathscr{P}_{\mathcal{T}}$, it only submits finitely many queries or assignment queries in its own interaction with A. By definition, $\mathscr{P}_{\mathcal{T}}$ only submits finitely many queries or assignment queries in each phase ψ of the simulated interaction. Because the protocols A_1 and A_2 are wait-free, $\mathscr{P}_{\mathcal{T}}$ can only end a finite number of phases by choosing a non-empty schedule β from $C_0\alpha(\psi)$ during which processes are *only* simulating A_1 and A_2. Every time that $\mathscr{P}_{\mathcal{T}}$ ends its phase by choosing a non-empty schedule β from $C_0\alpha(\psi)$ during which at least one process takes a step in the simulation of A, the prover $\mathscr{P}$ also ends its phase. It follows that $\mathscr{P}$ does not submit an infinite number of queries or assignment queries in any phase of its interaction with A. Therefore, the number of phases in the interaction between $\mathscr{P}$ and A will also be infinite. Thus, in all cases, $\mathscr{P}$ wins in its interaction against A.

Corollary 2. *If $(\mathcal{R}_1, \mathcal{R}_2)$ is a reduction from task $\mathcal{T}$ to task $\mathcal{S}$, there are anonymous protocols for $\mathcal{R}_1$ and $\mathcal{R}_2$, and there is an augmented extension-based proof that no anonymous protocol solves $\mathcal{T}$ in the NIS model, then there is an augmented extension-based proof that no anonymous protocol solves $\mathcal{S}$ in the NIS model.*

To see why this corollary is true, suppose that A_1 is an anonymous IS protocol for $\mathcal{R}_1$, A_2 is an anonymous NIS protocol for $\mathcal{R}_2$, and $\mathscr{P}_{\mathcal{T}}$ is an augmented extension-based prover that wins against any anonymous protocol claiming to solve $\mathcal{T}$ in the NIS model. If A is an anonymous NIS protocol claiming to solve S, then $\mathscr{P}_{\mathcal{T}}$ can also win against $A_2 \circ A \circ A_1$ by reaching two configurations C and C' such that some process q has terminated in C, some process q' has terminated in configuration C' with a different output value or has not terminated in configuration C', and the state of q in configuration C is the same as the state of q' in configuration C'. Construct $\mathscr{P}$ as in the proof of Theorem 1, except if $\mathscr{P}_{\mathcal{T}}$ wins by showing that $A_2 \circ A \circ A_1$ is not anonymous, then $\mathscr{P}$ wins by showing that A is not anonymous. Specifically, from the invariant and the fact that A_1 and A_2 are anonymous, $\mathscr{P}$ can show that it has reached two configurations D and D' of A such that q has terminated in D, q' has terminated in configuration D' with a different output value or has not terminated in configuration D', and the state of q in configuration D is the same as the state of q' in configuration D'.

We remark that the strategy described in the proof of Theorem 1 does not work for extension-based proofs, where $\mathscr{P}$ and $\mathscr{P}_{\mathcal{T}}$ can submit output queries instead of assignment queries. Suppose that $\mathscr{P}_{\mathcal{T}}$ submits an output query (C, P, y) in the simulated interaction, asking if there is a P-only schedule from C in which some process in P outputs the value y. Although there may be a schedule of A_2 from a configuration C_2 that leads to a configuration where some process in P outputs y, that process might not output y in the configuration reached by the same schedule from a different configuration C_2', where the only difference between C_2' and C_2 are the input values of some processes in P that have not yet taken any steps. Recall that the input values for the simulation of A_2 are the output values from the simulation of A. Thus $\mathscr{P}$ needs to determine if there is a schedule of A that leads to a single configuration where multiple processes output designated values. When $\mathscr{P}$ submits output queries to A, it cannot ask for a schedule where different values are output. Also, it cannot specify which process should output the desired value. It is unclear how $\mathscr{P}$ could determine whether such a schedule for A exists just using output queries. It was this limitation of extension-based proofs that lead us to define assignment queries and augmented extension-based proofs.

8 CONCLUSIONS

We showed that, in the NIS model, if there is a reduction (or anonymous reduction) from $\mathcal{T}$ to $\mathcal{S}$ and there is no augmented extension-based proof that (anonymous) protocols cannot solve $\mathcal{S}$, then there is no augmented extension-based proof that (anonymous) protocols cannot solve $\mathcal{T}$. Since there is no augmented extension-based proof that (anonymous) protocols cannot solve k-set agreement for $n > k \geq 2$ processes in the NIS model, it follows that there are no augmented extension-based proofs that k-leader election or k-test-and-set for $n > k \geq 2$ processes cannot be solved in the NIS model. Likewise, since there are anonymous protocols reducing weak symmetry breaking and $(2n-2)$-renaming to k-set agreement, there are no augmented extension-based proofs that anonymous protocols cannot solve weak symmetry breaking or $(2n-2)$-renaming.

An open problem that remains is to extend our results to general reductions from task $\mathcal{T}$ to task $\mathcal{S}$, allowing multiple instances of $\mathcal{S}$ to be used when solving $\mathcal{T}$. We would also like to generalize our results to tasks with infinitely many initial configurations. In addition, it would be interesting to understand whether having assignment queries rather than output queries gives the prover more power. Finally, is there a topological property of tasks that characterize when they have (augmented) extension-based proofs?

ACKNOWLEDGMENTS

We would like to thank the reviewers for their helpful comments. This work was supported by an Ontario Graduate Scholarship and the Natural Science and Engineering Research Council of Canada under grant RGPIN-2020-04178.

REFERENCES

[1] Yehuda Afek, Hagit Attiya, Danny Dolev, Eli Gafni, Michael Merritt, and Nir Shavit. 1993. Atomic Snapshots of Shared Memory. *JACM* 40, 4 (1993), 873–890. https://doi.org/10.1145/153724.153741

[2] Dan Alistarh, James Aspnes, Faith Ellen, Rati Gelashvili, and Leqi Zhu. 2019. Why Extension-based Proofs Fail. In *Proceedings of the 51'st Annual ACM Symposium on Theory of Computing (STOC)*. 986–996. https://doi.org/10.1145/3313276.3316407

[3] Dan Alistarh, James Aspnes, Faith Ellen, Rati Gelashvili, and Leqi Zhu. 2020. Brief Announcement: Why Extension-Based Proofs Fail. In *Proceedings of the 39'th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. 54–56. https://doi.org/10.1145/3382734.3405743

[4] Dan Alistarh, Faith Ellen, and Joel Rybicki. 2021. Wait-free Approximate Agreement on Graphs. In *Proceedings of the 28'th International Colloquium on Structural Information and Communication Complexity (SIROCCO)*. To appear.

[5] Hagit Attiya, Armando Castañeda, and Sergio Rajsbaum. 2020. Locally Solvable Tasks and the Limitations of Valency Arguments. In *Proceedings of the 24'th International Conference on Principles of Distributed Systems (OPODIS)*. 18:1–18:16. https://doi.org/10.4230/LIPIcs.OPODIS.2020.18

[6] Elizabeth Borowsky and Eli Gafni. 1993. Generalized FLP Impossibility Result for t-resilient Asynchronous Computations. In *Proceedings of the 25'th Annual ACM Symposium on Theory of Computing (STOC)*. 91–100. https://doi.org/10.1145/167088.167119

[7] Elizabeth Borowsky and Eli Gafni. 1993. Immediate Atomic Snapshots and Fast Renaming (Extended Abstract). In *Proceedings of the 12'th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. 41–51. https://doi.org/10.1145/164051.164056

[8] Elizabeth Borowsky and Eli Gafni. 1997. A Simple Algorithmically Reasoned Characterization of Wait-Free Computations. In *Proceedings of the 16'th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. 189–198. https://doi.org/10.1145/259380.259439

[9] Armando Castañeda and Sergio Rajsbaum. 2010. New Combinatorial Topology Bounds for Renaming: the Lower Bound. *Distributed Comput.* 22, 5-6 (2010), 287–301. https://doi.org/10.1007/s00446-010-0108-2

[10] Armando Castañeda, Sergio Rajsbaum, and Matthieu Roy. 2018. Convergence and Covering on Graphs for Wait-free Robots. *J. Braz. Comput. Soc.* 24, 1 (2018), 1:1–1:15. https://doi.org/10.1186/s13173-017-0065-8

[11] Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *JACM* 32, 2 (1985), 374–382. https://doi.org/10.1145/3149.214121

[12] Eli Gafni. 2006. Read-Write Reductions. In *Proceedings of the 8'th International Conference on Distributed Computing and Networking (ICDCN)*. 349–354. https://doi.org/10.1007/11947950_38

[13] Eli Gafni, Achour Mostéfaoui, Michel Raynal, and Corentin Travers. 2009. From Adaptive Renaming to Set Agreement. *Theor. Comput. Sci.* 410, 14 (2009), 1328–1335. https://doi.org/10.1016/j.tcs.2008.05.016

[14] Eli Gafni, Sergio Rajsbaum, and Maurice Herlihy. 2006. Subconsensus Tasks: Renaming Is Weaker Than Set Agreement. In *Proceedings of the 20'th International Symposium on Distributed Computing (DISC)*. 329–338. https://doi.org/10.1007/11864219_23

[15] Maurice Herlihy, Dmitry N. Kozlov, and Sergio Rajsbaum. 2013. *Distributed Computing Through Combinatorial Topology*. Morgan Kaufmann.

[16] Maurice Herlihy and Nir Shavit. 1999. The Topological Structure of Asynchronous Computability. *JACM* 46, 6 (1999), 858–923. https://doi.org/10.1145/331524.331529

[17] Gunnar Hoest and Nir Shavit. 2006. Toward a Topological Characterization of Asynchronous Complexity. *SIAM J. Comput.* 36, 2 (2006), 457–497. https://doi.org/10.1137/S0097539701397412

[18] Michael E. Saks and Fotios Zaharoglou. 2000. Wait-Free k-Set Agreement is Impossible: The Topology of Public Knowledge. *SIAM J. Comput.* 29, 5 (2000), 1449–1483. https://doi.org/10.1137/S0097539796307698